

Лекция 4. Частотная область и многомасштабный анализ

Синусоиды и спектр, ДПФ и FFT, теорема о свёртке, пирамиды, JPEG, вейвлеты

Александр Савельев · Курс «Анализ изображений», 3 курс бакалавриата

План лекции

1. **Изображение как сумма волн** — синусоида, её параметры, 2D-решётки 12 мин
2. **Дискретное преобразование Фурье** — комплексные числа, ДПФ, спектр, фаза, FFT 18 мин
3. **Теорема о свёртке и фильтрация** — почему box звенит, ФНЧ/ФВЧ, удаление помех 20 мин
4. **Пирамиды изображений** — гауссова, лапласова, блендинг 13 мин
5. **JPEG и вейвлеты** — где именно в сжатии используются частоты 15 мин

Сквозная линия

Любое изображение — **сумма волн разной частоты**. Низкие частоты — общая форма и плавные переходы, высокие — края, текстура, шум. Сглаживание, уменьшение, пирамиды и JPEG — это разные способы **обращаться с высокими частотами**.

Короткие листинги на Python/NumPy показывают, что каждая формула — это 2–5 строк кода.

Часть 1

Изображение как сумма волн

Определение

Частота — сколько раз узор повторяется на единицу длины. В изображениях её меряют в **периодах на пиксель** (или периодах на всю ширину картинки).

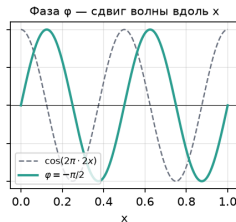
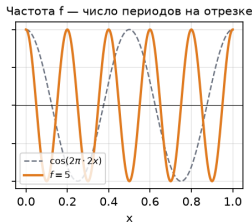
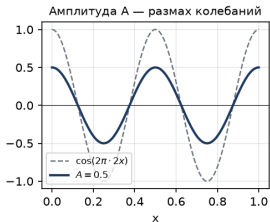
Частотное представление — запись изображения как **суммы волн** (синусоид) разных частот, направлений и амплитуд.

Что покажем

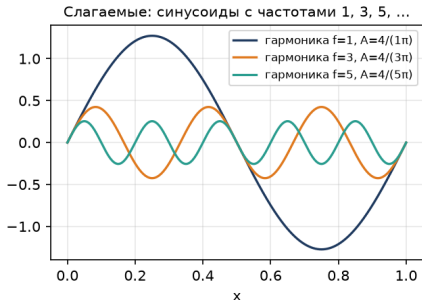
- что такое синусоида и какие у неё три параметра;
- что сумма простых волн может дать любой сигнал — даже ступеньку;
- как выглядит волна на плоскости и как из таких волн собирается фотография.

$$s(x) = A \cos(2\pi f x + \varphi)$$

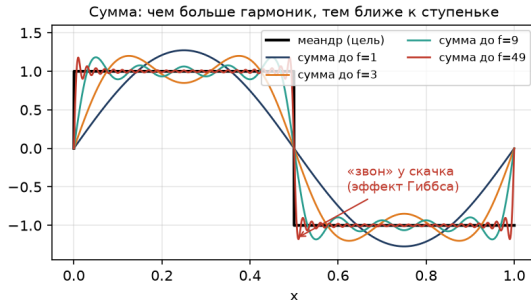
- x — координата (в 1D — номер пикселя в строке); $s(x)$ — яркость в этой точке;
- A — **амплитуда**: насколько сильно яркость отклоняется от среднего (контраст волны);
- f — **частота**: сколько периодов на единицу длины; период $T = 1/f$ — длина одной «волны»;
- φ — **фаза** (в радианах): на сколько волна сдвинута вдоль x ; сдвиг на $\varphi =$ сдвиг на $\varphi/(2\pi f)$ пикселей.



Пример: $f = 0.25$ периода/пиксель $\Rightarrow T = 4$ пикселя: светлая полоса повторяется каждые 4 пикселя. Самая быстрая волна на пиксельной сетке — $f = 0.5$: «светлый, тёмный, светлый, ...» (период 2 пикселя).



Что на рисунке. Ступенчатый сигнал («меандр») собран из синусоид с частотами 1, 3, 5, ... и амплитудами $\frac{4}{\pi}$, $\frac{4}{3\pi}$, $\frac{4}{5\pi}$, ... Чем больше слагаемых, тем **круче** получаются фронты. Это **ряд Фурье** (Ж. Фурье, 1807).

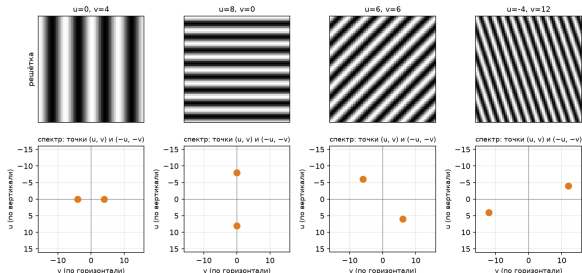


Два вывода на всю лекцию. (1) **Резкий край = много высоких частот**: без них ступенька становится плавной. (2) Если высокие частоты резко «отрезать», у скачка остаются волны — **эффект Гиббса**. Это и есть «звон» у box-фильтра и идеального фильтра.

$$g[m, n] = \cos\left(2\pi\left(\frac{u m}{M} + \frac{v n}{N}\right)\right)$$

- m, n — номер строки и столбца;
 $M \times N$ — размер картинки;
- u — сколько периодов укладывается **по вертикали** (по высоте M);
- v — сколько периодов **по горизонтали** (по ширине N);
- пара (u, v) — **вектор частоты**:
полосы перпендикулярны ему,
длина $\sqrt{u^2 + v^2}$ — насколько они частые.

Внизу — **спектр** решётки: одна волна
= **две точки** (u, v) и $(-u, -v)$.



$|частота| \leq 4$: 49 решёток



$|частота| \leq 12$: 441 решёток



$|частота| \leq 32$: 3209 решёток



$|частота| \leq 80$: 20081 решёток



все 262 144 решётки



Опыт. Картинку 512×512 можно точно записать как сумму $512 \cdot 512 = 262\,144$ решёток (столько же, сколько пикселей). Оставим только решётки с **малой** длиной вектора частоты $\sqrt{u^2 + v^2} \leq R$ и сложим их.

Что видно. 49 самых медленных решёток уже дают **общее распределение света**; 3 тыс. (1.2 %) — узнаваемую сцену; 20 тыс. (8 %) — почти всё. Высокие частоты — это **мелкие детали, края и шум**. На этом стоят и сглаживание, и сжатие.

Часть 2

Дискретное преобразование Фурье

Определение

Дискретное преобразование Фурье (ДПФ) — правило, которое по N отсчётам сигнала находит, **сколько в нём каждой из N волн:** для каждой частоты — амплитуду и фазу.

Спектр — результат ДПФ: таблица «частота $\rightarrow$ амплитуда и фаза». Для картинки $M \times N$ — тоже таблица $M \times N$.

Что покажем

- минимум про комплексные числа — зачем они здесь;
- формулу ДПФ в 1D и 2D и как её читать;
- как смотреть на спектр и что в нём видно;
- почему фаза важнее амплитуды и как ДПФ считают быстро (FFT).

Зачем. Каждая волна описывается **двумя** числами — амплитудой и фазой. Комплексное число хранит ровно два числа.

$$z = a + ib = |z| e^{i\varphi}, \quad e^{i\theta} = \cos \theta + i \sin \theta$$



Формула Эйлера: вращение = синус + косинус



- i — мнимая единица, $i^2 = -1$; a — действительная часть, b — мнимая;
- $|z| = \sqrt{a^2 + b^2}$ — **длина стрелки** = амплитуда волны;
- $\varphi = \text{atan2}(b, a)$ — **угол стрелки** = фаза волны;
- $e^{i\theta}$ — стрелка длины 1 под углом θ (формула Эйлера); при умножении модули перемножаются, углы складываются.

Волна та же. Меняется только запись. Возьмём

$$s(x) = 2 \cos(2\pi fx + \pi/3) \quad \text{то есть} \quad A = 2, \varphi = \pi/3.$$

Тождество $\cos(\alpha + \varphi) = \cos \alpha \cos \varphi - \sin \alpha \sin \varphi$ даёт

$$s(x) = \underbrace{A \cos \varphi}_a \cos(2\pi fx) - \underbrace{A \sin \varphi}_b \sin(2\pi fx).$$

Числа примера

$$a = 2 \cos(\pi/3) = 1, \quad b = 2 \sin(\pi/3) = \sqrt{3}.$$

$$2 \cos(2\pi fx + \pi/3) = \cos(2\pi fx) - \sqrt{3} \sin(2\pi fx).$$

Что изменилось

Слева параметры — **амплитуда и фаза**. Справа — **два веса** перед несдвинутыми косинусом и синусом. Сдвинутый косинус собирается из них в нужной пропорции. Яркость в точке x та же.

Числа a и b удобно хранить вместе: $z = a + ib$. В примере $z = 1 + i\sqrt{3}$. На яркость это не влияет: мы только **упаковали** два коэффициента.

Запись	Два числа
Амплитуда и фаза	$A = 2, \varphi = \pi/3$
Re и Im части	$a = 1, b = \sqrt{3}$

Переход в обе стороны:

$$a = A \cos \varphi, \quad b = A \sin \varphi$$

$$A = \sqrt{a^2 + b^2}, \quad \varphi = \text{atan2}(b, a)$$

$$(A, \varphi) \longleftrightarrow (a, b) \longleftrightarrow z = a + ib$$

Мы не превратили изображение в «мнимое». Мы поменяли способ хранения параметров волны.

Формула Эйлера

Та же упаковка короче:

$$a + ib = A \cos \varphi + iA \sin \varphi = Ae^{i\varphi}.$$

$Ae^{i\varphi}$ — **не сама волна**, а коэффициент с амплитудой и фазой. Частота по-прежнему в $\cos(2\pi fx)$ и $\sin(2\pi fx)$.

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-2\pi i \frac{kn}{N}}, \quad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{+2\pi i \frac{kn}{N}}$$

- $x[n]$ — сигнал: N чисел (например, яркости строки), $n = 0..N-1$ — номер отсчёта;
- $k = 0..N-1$ — номер частоты: волна, которая делает k периодов на всю длину N ;
- $X[k]$ — комплексное число: $|X[k]|$ — сколько этой волны в сигнале, $\arg X[k]$ — её фаза;
- справа — **обратное ДПФ**: тот же сигнал как сумма N волн; ничего не теряется.

Смысл одной строкой

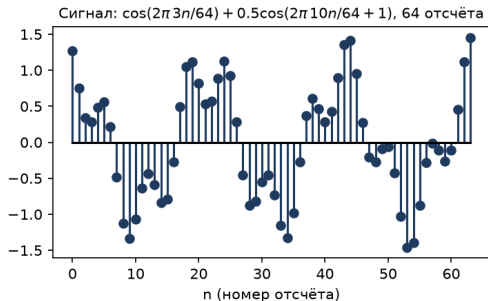
$X[k]$ — **скалярное произведение** сигнала с волной частоты k : «насколько сигнал похож на эту волну». Похож — большое число, не похож — около нуля.

Полезные факты

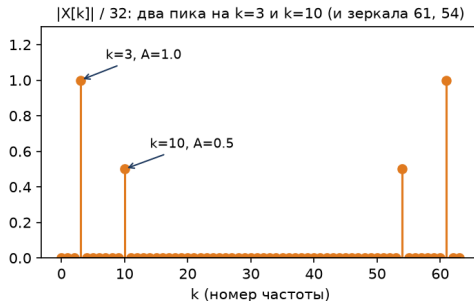
$X[0] = \sum_n x[n]$ — сумма (среднее $\times N$). Для действительного x : $X[N-k] = \overline{X[k]}$ — вторая половина спектра — **зеркало** первой. Амплитуда волны $A_k = 2 |X[k]| / N$.

Пример: две волны → два пика

Дискретное преобразование Фурье



Слева — 64 отсчёта суммы двух волн: 3 периода с амплитудой 1 и 10 периодов с амплитудой 0.5 и сдвигом фазы. Глазом в этой «каше» волны не разделить.



Справа — $|X[k]|$, нормированное на $N/2$: ровно **два пика** на $k=3$ и $k=10$ с высотами 1 и 0.5, всё остальное — нули. Пики на 61 и 54 — зеркала ($64 - 3, 64 - 10$). Фазу 1 рад можно прочесть как $\arg X[10]$.

$$F[u, v] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} I[m, n] e^{-2\pi i \left(\frac{um}{M} + \frac{vn}{N} \right)}$$

- $I[m, n]$ — изображение $M \times N$ (m — строка, n — столбец);
- $F[u, v]$ — сколько в изображении решётки с частотами u (по вертикали) и v (по горизонтали), **как на слайде с решётками**; $|F|$ — амплитуда, $\arg F$ — фаза;
- обратное ДПФ — та же формула со знаком $+$ в показателе и множителем $\frac{1}{MN}$.

Разделимость

Экспоненту можно разбить:

$e^{-2\pi i(\cdot)} = e^{-2\pi i u m / M} \cdot e^{-2\pi i v n / N}$. Значит,
2D-ДПФ = **1D-ДПФ всех строк**, потом
1D-ДПФ всех столбцов — как разделимое
ядро в лекции 3.

Не путать

$F[0, 0] = \sum I$ — «постоянная составляющая»,
средняя яркость $\times MN$. Обычно она в тысячи
раз больше остальных коэффициентов —
поэтому спектр показывают в
логарифмическом масштабе.

```
import numpy as np

F = np.fft.fft2(I)          # 2D-ДПФ, массив MxN
F = np.fft.fftshift(F)      # (0,0) - в центр
A = np.abs(F)               # амплитуды |F|
phi = np.angle(F)           # фазы от -pi до pi
S = np.log1p(A)              # показ: log(1+|F|)

# обратно: вернуть порядок частот
F0 = np.fft.ifftshift(F)
I_back = np.fft.ifft2(F0).real
assert np.allclose(I, I_back) # без потерь
```

- `fft2` считает ровно формулу с прошлого слайда (быстрым алгоритмом — в конце части).
- `fftshift` переставляет четверти массива: без неё нулевая частота в углу $[0, 0]$. Это только **для удобства просмотра**.
- `log1p = log(1 + x)`: сжимает разброс от 10^5 до 10^0 , иначе видна одна точка.
- `.real`: мнимая часть после обратного ДПФ — ошибки округления $\sim 10^{-16}$.

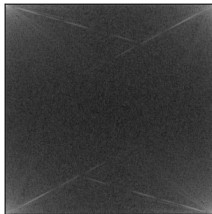
Изображение I



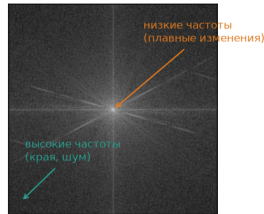
$|F|$ как есть: видна одна точка — всё чёрное



$\log(1+|F|)$: нули частот в углах



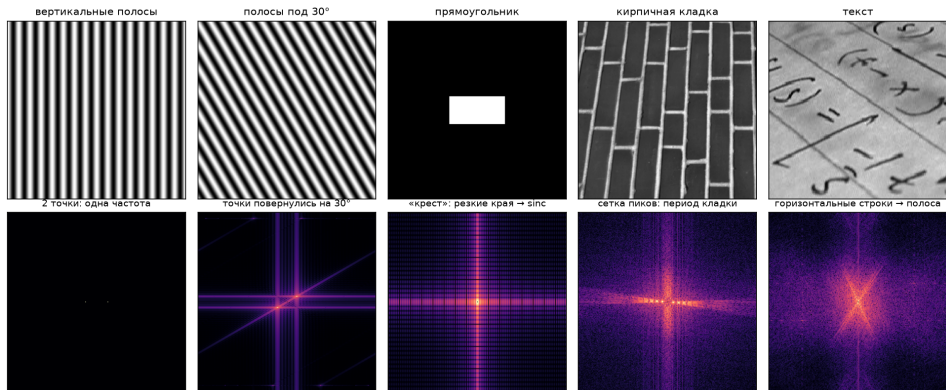
+ fftshift: нулевая частота в центре



Без логарифма видна только $F[0, 0]$ — средняя яркость; остальное на её фоне ≈ 0 .

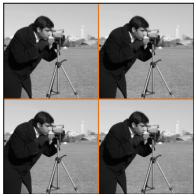
С логарифмом и сдвигом: центр — **низкие частоты**, к краям — **высокие**. Расстояние от центра = частота, направление от центра = направление волны.

Типичная картина: яркий центр и спад к краям — у фотографий амплитуда убывает примерно как $1/|f|$: **низких частот больше**.

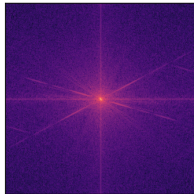


Полосы — пара точек на расстоянии, равном частоте; повернули полосы — **повернулись точки**. **Резкие края** прямоугольника — длинные «хвосты» поперёк краёв (как у меандра: ступенька = много частот). **Периодическая текстура** (кладка) — сетка пиков. **Текст** — полоса поперёк строк: шаг строк виден как частота.

ДПФ «думает», что картинка повторяется: швы на стыках



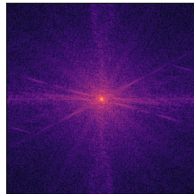
спектр: яркий «крест» от швов, а не от сцены



умножили на окно Ханна: края плавно гаснут в 0



спектр без креста



ДПФ раскладывает по волнам, которые **повторяются** с периодом M и N , т. е. молча считает, что картинка «склеена» с копиями себя — граничное условие **wrap** из лекции 3. Верх снимка (светлое небо) стыкуется с низом (тёмная трава) — на стыке **резкий шов**, которого в сцене нет. Ступенька даёт много частот $\Rightarrow$ яркий «крест» по осям.

$$w[m, n] = h[m] h[n], \quad h[n] = \frac{1}{2} \left(1 - \cos \frac{2\pi n}{N-1} \right)$$

- $h[n]$ — **окно Ханна**: 0 на краях, 1 в середине, плавно;
- спектр считаем от $I \cdot w$: края гаснут, шва нет, крест исчезает — остаётся структура сцены.

А: оператор



В: астронавт



амплитуда В + фаза А



амплитуда А + фаза В



Опыт. Посчитаем ДПФ двух снимков, возьмём **амплитуды от одного, фазы от другого** и сделаем обратное ДПФ: $I = \mathcal{F}^{-1}[|F_B| e^{i \arg F_A}]$.

Результат. Мы видим того, **чья фаза**. Амплитуды у всех фотографий похожи ($\sim 1/|f|$), а **где** стоят края и объекты, кодирует фаза: она говорит, как волны выстроены относительно друг друга. Вывод: смотреть на $|F|$ полезно, но выбрасывать фазу нельзя.

Проблема. По формуле каждый из N коэффициентов — сумма N слагаемых: N^2 умножений. **Идея FFT** (Кули–Тьюки, 1965): разбить сигнал на чётные и нечётные отсчёты и посчитать два ДПФ половинной длины.

$$X[k] = E[k] + W^k O[k], \quad X\left[k + \frac{N}{2}\right] = E[k] - W^k O[k]$$

- $E[k], O[k]$ — ДПФ длины $N/2$ от чётных $x[0], x[2], \dots$ и нечётных $x[1], x[3], \dots$;
- $W = e^{-2\pi i/N}$ — поворот на $1/N$ оборота; одна пара E, O даёт **два** коэффициента;
- делим пополам $\log_2 N$ раз $\Rightarrow O(N \log N)$ вместо $O(N^2)$.

Сколько это для 512×512

2D-ДПФ «в лоб», $(MN)^2$	$6.9 \cdot 10^{10}$
строки, потом столбцы	$2.7 \cdot 10^8$
FFT, $MN \log_2(MN)$	$4.7 \cdot 10^6$

Выигрыш FFT — в $\sim 15\,000$ раз против «в лоб». Быстрее всего FFT работает, когда размеры — произведения малых простых чисел (512, 480, 1000); для простого размера 509 — заметно медленнее.

Часть 3

Теорема о свёртке и фильтрация

Определение

Теорема о свёртке: свёртка в пространстве = **поэлементное умножение спектров**.

Частотная характеристика фильтра $H[u, v]$ — ДПФ его ядра: **во сколько раз** фильтр умножает волну с частотой (u, v) . $|H| = 1$ — пропускает, $|H| \approx 0$ — гасит, $H < 0$ — переворачивает.

Что покажем

- теорему и её картинку;
- почему box «звенит», а Гаусс — нет (долг лекции 3);
- фильтры низких и высоких частот, удаление периодической помехи;
- когда свёртку выгодно считать через FFT.

$$\mathcal{F}\{I * h\}[u, v] = F[u, v] \cdot H[u, v] \quad \Longleftrightarrow \quad I * h = \mathcal{F}^{-1}\{F \cdot H\}$$

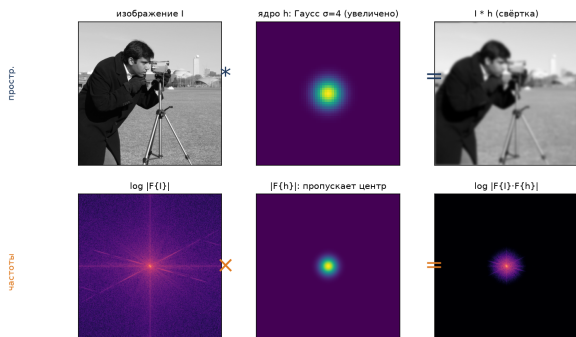
- $\mathcal{F}$ — ДПФ, $\mathcal{F}^{-1}$ — обратное ДПФ; $F = \mathcal{F}\{I\}$ — спектр изображения, $H = \mathcal{F}\{h\}$ — спектр ядра (ядро дополнено нулями до $M \times N$);
- « $\cdot$ » — **поэлементное** умножение: каждый коэффициент $F[u, v]$ умножается на своё число $H[u, v]$.

Почему так

Волна — «собственный вектор» свёртки: свернём решётку с любым ядром — получим **ту же решётку**, только с другой амплитудой и фазой. Множитель и есть $H[u, v]$. Изображение — сумма решёток, свёртка линейна $\Rightarrow$ каждую решётку умножаем на своё H .

Тонкость: свёртка циклическая

Через ДПФ получается свёртка с границей **wrap**: то, что «вылезло» за правый край, приходит слева. Чтобы получить обычную свёртку, картинку дополняют (reflect или нули) хотя бы на k пикселей и потом обрезают.

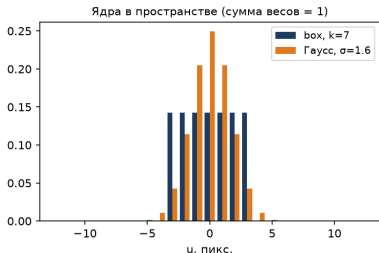


Верхний ряд — то, что делали в лекции 3: свёртка с гауссовым ядром $\sigma = 4$, получилось размытое изображение.

Нижний ряд — то же самое в частотах. Спектр ядра $|H|$ — яркое пятно в центре: **низкие частоты пропускает** ($H \approx 1$), высокие **гасит** ($H \approx 0$). Произведение — спектр картинки, «обрезанный» до центра.

Обратное ДПФ нижнего правого спектра даёт **в точности** верхнее правое изображение.

Поэтому сглаживающие фильтры называют фильтрами **низких частот** (ФНЧ, low-pass).



$$H_{\text{box}}(f) = \frac{\sin(\pi k f)}{k \sin(\pi f)}$$

- k — ширина box, f — частота в периодах/пиксель;
- нули при $f = 1/k, 2/k, \dots$; между ними — **отрицательные лепестки**: волна с $H < 0$ выходит **перевернутой** — светлые полосы становятся тёмными. Это и есть «звон».



$$H_{\text{Гаусс}}(f) = e^{-2\pi^2 \sigma^2 f^2}$$

- σ — параметр ядра в пикселях;
- спектр Гаусса — **снова Гаусс**: всегда > 0 , монотонно убывает — ничего не переворачивает;
- ширина в частотах $\sim 1/(2\pi\sigma)$: **шире ядро — уже полоса пропускания**.

Фильтр можно задавать **сразу в частотах**: нарисовать маску $H[u, v]$ и умножить на спектр. $D(u, v)$ — расстояние от точки спектра до центра (после `fftshift`), D_0 — **частота среза**: где фильтр «начинает гасить».

Идеальный

$$H = \begin{cases} 1, & D \leq D_0 \\ 0, & D > D_0 \end{cases}$$

Резкий обрыв: как у меандра, обрыв частот даёт **сильный звон** (эффект Гиббса).

Баттерворт

$$H = \frac{1}{1 + (D/D_0)^{2n}}$$

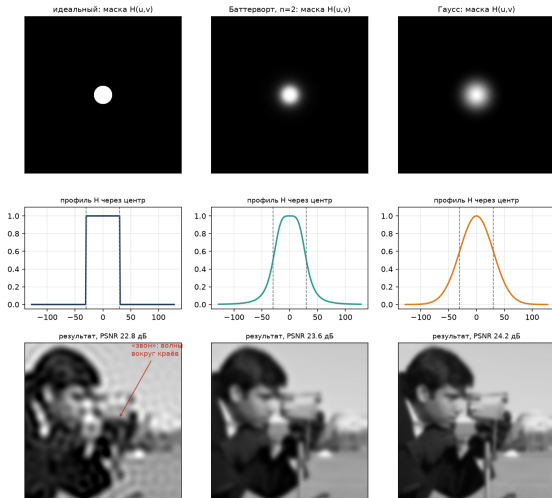
n — порядок: $n = 1$ — плавно, $n \rightarrow \infty$ — идеальный. При $n = 2$ звон почти незаметен. На $D = D_0$ всегда $H = 1/2$.

Гауссов

$$H = e^{-D^2/(2D_0^2)}$$

Самый плавный: **звона нет совсем**. Это просто Гаусс из лекции 3, заданный в частотах: $D_0 \leftrightarrow N/(2\pi\sigma)$.

D и D_0 измеряются в «периодах на всю картинку» (как u, v); $D_0 = 30$ для 512×512 — это волны с периодом от ≈ 17 пикселей и длиннее.



Одинаковая частота среза $D_0 = 30$,
разная форма маски.

Верх — маска, **середина** — её профиль
через центр, **низ** — фрагмент
результата.

Идеальный: вокруг фигуры и штатива
волны-«ореолы» — звон; PSNR хуже
всех.

Баттерворт $n = 2$: звон еле заметен.

Гаусс: звона нет, размытие самое
«естественное».

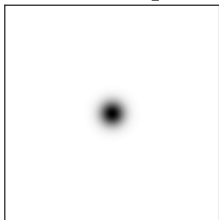
Правило

Резкая граница в частотах $\Rightarrow$ волны в
пространстве, и наоборот. На практике
берут **гладкие маски**.

$$H_{\text{ФВЧ}} = 1 - H_{\text{ФНЧ}}, \quad H_{\text{полос}} = H_{\text{ФНЧ}}(D_1) - H_{\text{ФНЧ}}(D_0), \quad D_1 > D_0$$

- ФВЧ (high-pass) — оставляет то, что ФНЧ выбросил: **края, текстуру, шум**; среднюю яркость ($D = 0$) обнуляет;
- полосовой — оставляет только частоты **между** D_0 и D_1 — детали определённого размера.

маска ФВЧ: $1 - H_{\text{Гаусс}}$



ФВЧ: остались края и текстура $I + 1.5 \cdot \text{ФВЧ} = \text{повышение резкости}$



полосовой: средние частоты

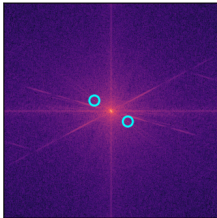


Третья картинка — $I + 1.5 \cdot \text{ФВЧ}(I)$: это **unsharp masking** из лекции 3 ($I + \alpha(I - G * I)$), записанный в частотах: $H = 1 + \alpha(1 - H_{\text{Гаусс}})$ — усиливает высокие частоты в $1 + \alpha$ раз.

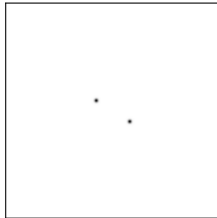
помеха-«сетка», PSNR 15.1 дБ



в спектре: два ярких пика



маска: «дырки» на месте пиков



после фильтра, PSNR 49.2 дБ



Задача. На снимок наложена косая «сетка» (так выглядят наводки в сканах, муар от раstra печати, помехи матрицы). В пространстве её не отделить от деталей — она везде.

Алгоритм (режекторный фильтр)

1. Спектр $\log |F|$; помеха = **одна волна** $\Rightarrow$ два ярких пика $\pm(u_0, v_0)$.
2. Маска $H = 1$ везде, кроме маленьких «дырок» (гауссовых, радиус 3) на пиках.
3. $\mathcal{F}^{-1}\{F \cdot H\}$: помеха ушла, детали остались. PSNR 15 $\rightarrow$ 49 дБ.

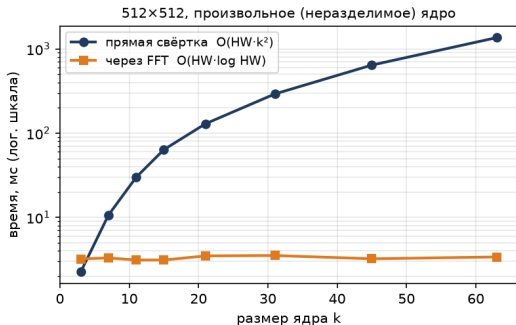
```
M, N = I.shape
# координаты точек спектра от центра
u = np.arange(M) - M // 2
v = np.arange(N) - N // 2
V, U = np.meshgrid(v, u) # V - горизонталь
D = np.sqrt(U**2 + V**2) # D(u, v)

D0 = 30
H_lp = np.exp(-D**2 / (2 * D0**2)) # ФНЧ
H_hp = 1 - H_lp                    # ФВЧ

F = np.fft.fftshift(np.fft.fft2(I))
def back(G):                        # в пространство
    G = np.fft.ifftshift(G)
    return np.fft.ifft2(G).real

low, high = back(F * H_lp), back(F * H_hp)
assert np.allclose(low + high, I)
```

- Маска строится на сетке размера спектра с центром в $(M/2, N/2)$ — спектр сдвинут `fftshift`.
- $F * H_{lp}$ — поэлементное умножение: это и есть теорема о свёртке.
- Перед обратным ДПФ — `ifftshift`: вернуть частоты на места.
- Последняя строка: $\text{ФНЧ} + \text{ФВЧ} = 1 \Rightarrow \text{«низкое»} + \text{«высокое»} = \text{исходное}$. Это основа лапласовой пирамиды (часть 4).



SciPy: `ndimage.convolve` против `signal.fftconvolve`, ядро $k \times k$ из случайных чисел.

Стоимость. Прямая свёртка — $O(MN k^2)$, растёт с ядром. Через FFT — три преобразования и умножение: $O(MN \log(MN))$, от k **почти не зависит**.

Практическое правило

- Маленькое ядро ($k \lesssim 7$) или **разделимое** (Гаусс, box) — прямая свёртка: она проще и не требует дополнения краёв.
- Большое **неразделимое** ядро ($k \gtrsim 11$), например размытие от движения или PSF объектива — FFT, выигрыш в 10–100 раз.
- Фильтр задан **маской в частотах** (режекторный, Баттерворт) — только через FFT.

Часть 4

Пирамиды изображений

Определение

Пирамида изображения — набор его копий, где каждая следующая **в 2 раза меньше** по каждой стороне (и правильно сглажена перед уменьшением).

Зачем: объекты бывают разного размера; на маленьком уровне крупный объект становится «нормального» размера, а алгоритм работает в 4, 16, 64 раза быстрее.

Что покажем

- гауссову пирамиду — последовательность всё более грубых копий;
- лапласову пирамиду — разложение на полосы частот без потерь;
- классический пример — бесшовное «яблоко-апельсин».

$$G_0 = I, \quad G_{l+1} = \downarrow_2 (G_\sigma * G_l), \quad l = 0, 1, 2, \dots$$

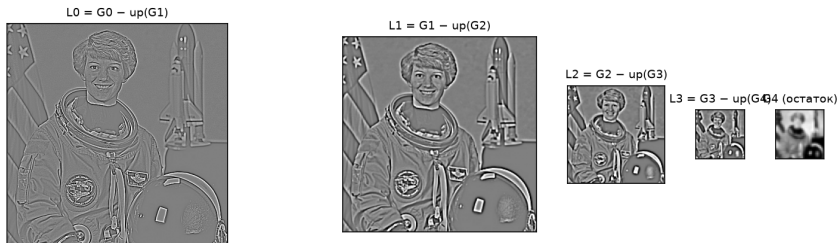
- G_l — уровень l , размер $\frac{M}{2^l} \times \frac{N}{2^l}$; $G_\sigma *$ — сглаживание ядром $\frac{1}{16} [1, 4, 6, 4, 1]$ по строкам и столбцам ($\sigma \approx 1$);
- $\downarrow_2$ — взять каждый второй пиксель по строкам и столбцам; сглаживание перед ним — **ФНЧ**: убирает детали мельче, чем покажет уменьшенная копия.



Каждый уровень — ФНЧ с вдвое меньшей частотой среза. Память: $1 + \frac{1}{4} + \frac{1}{16} + \dots = \frac{4}{3}$ исходной. Один шаг — `cv2.pyrDown`.

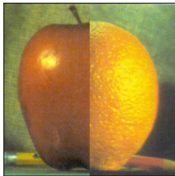
$$L_l = G_l - \uparrow_2 G_{l+1}, \quad l = 0, \dots, K-1; \quad L_K = G_K$$

- $\uparrow_2$ — увеличить в 2 раза: вставить нули между пикселями и сгладить тем же ядром ($\times 4$) — `cv2.pyrUp`;
- $\uparrow_2 G_{l+1}$ — «что можно восстановить по уровню поменьше» (размыто); L_l — **разница**: детали, которые уровень $l+1$ уже не хранит; K — число уровней.

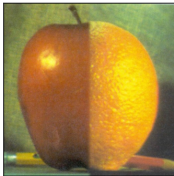


Серый — ноль, светлое и тёмное — положительные и отрицательные детали. L_0 — самые мелкие: волосы, швы, текстура ткани; L_3 — крупные контуры. **Хранить в float**: значения L_l бывают отрицательными.

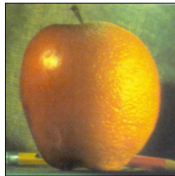
1) склейка по маске:
резкий шов



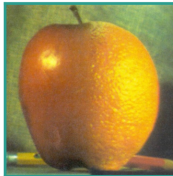
2) маска размыта узко ($\sigma=4$):
шов по цвету виден



3) маска размыта широко ($\sigma=60$):
полоса «полупрозрачной» смеси



4) лапласова пирамида:
шва нет, текстуры чёткие



Задача — склеить левую половину яблока с правой половиной апельсина. Узкий переход (1, 2) — **шов**; широкий (3) — текстура **двоится**. Нужен **свой переход для каждого масштаба**: цвет смешивать широко, текстуру — узко.

Алгоритм (Burt, Adelson, 1983)

1. Лапласовы пирамиды L_I^A , L_I^B двух картинок; гауссова пирамида G_I^M маски M (1 — берём A , 0 — B).
2. На каждом уровне: $L_I^S = G_I^M \cdot L_I^A + (1 - G_I^M) \cdot L_I^B$.
3. Собрать S из L_I^S по формуле восстановления.

На уровне I маска размыта на $\sim 2^I$ пикселей.

Разложение

Вход: изображение I , число уровней K .

Выход: $G_0, \dots, G_K$ и $L_0, \dots, L_K$.

1. $G_0 \leftarrow I$
2. для $l = 0, \dots, K - 1$: $G_{l+1} \leftarrow \downarrow_2 (g * G_l)$
3. для $l = 0, \dots, K - 1$: $L_l \leftarrow G_l - \uparrow_2 G_{l+1}$
4. $L_K \leftarrow G_K$

Сборка

1. $S_K \leftarrow L_K$
2. для $l = K - 1, \dots, 0$: $S_l \leftarrow L_l + \uparrow_2 S_{l+1}$
3. вернуть S_0

- $g = \frac{1}{16} [1, 4, 6, 4, 1]$ по строкам и столбцам;
- $(\downarrow_2 X)[m, n] = X[2m, 2n]$ — каждый второй пиксель;
- $\uparrow_2 X = 4(g * Y)$, где $Y[2m, 2n] = X[m, n]$, остальные $Y = 0$;
- размер $G_l = \frac{M}{2^l} \times \frac{N}{2^l}$.

Почему сборка точная

Индукция снизу: $S_K = G_K$. Если $S_{l+1} = G_{l+1}$, то

$$S_l = (G_l - \uparrow_2 G_{l+1}) + \uparrow_2 G_{l+1} = G_l.$$

Значит, $S_0 = G_0 = I$ — **при любом** ядре g .
Считать в вещественных числах: L_l бывают отрицательными.

Блендинг (Burt, Adelson, 1983)

Вход: изображения A , B одного размера; маска M , $M[m, n] \in [0, 1]$; число уровней K .

Выход: склейка S .

1. $\{L_l^A\}$, $\{L_l^B\}$ — лапласовы пирамиды A и B
2. $\{G_l^M\}$ — гауссова пирамида маски M
3. для $l = 0, \dots, K$:

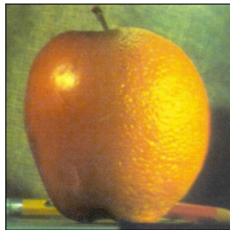
$$L_l^S \leftarrow G_l^M \odot L_l^A + (1 - G_l^M) \odot L_l^B$$

4. $S \leftarrow$ сборка $\{L_l^S\}$
5. $S \leftarrow \min(\max(S, 0), 255)$

- $\odot$ — поэлементное умножение;
- $M = 1$ — берём A , $M = 0$ — берём B ;
- G_l^M — маска, размытая на $\sim 2^l$ пикселей исходного размера.

На мелких уровнях переход узкий — текстура не двоится. На грубых широкий — цвет меняется плавно, шва нет.

результат S



Часть 5

JPEG и вейвлеты

Определение

Сжатие с потерями — уменьшение размера файла за счёт того, что **глаз почти не замечает**; восстановленная картинка близка к исходной, но не равна ей.

Идея JPEG: в маленьком блоке 8×8 высоких частот мало, и глаз к ним менее чувствителен — большинство их коэффициентов можно **округлить до нуля**.

Что покажем

- шаги JPEG и где именно теряется информация;
- DCT — «ДПФ для блоков»;
- размер файла против качества, артефакты;
- вейвлеты и JPEG 2000 — идея без деталей.



- **Без потерь:** смена цветового пространства, разбиение, DCT (обратимо), энтропийное кодирование.
- **С потерями:** только два шага (оранжевые). Параметр «качество» 1..100 управляет вторым.
- Декодер проходит цепочку **в обратном порядке**: коэффициенты $\times Q \rightarrow$ обратное DCT $\rightarrow$ увеличить Cb, Cr $\rightarrow$ RGB.
- Всё, что мы изучили сегодня, здесь есть: частоты, ФНЧ, прореживание, восприятие.

$$Y = 0.299R + 0.587G + 0.114B, \quad C_b = 128 + 0.564(B - Y), \quad C_r = 128 + 0.713(R - Y)$$

- $R, G, B \in [0, 255]$; Y — **яркость** (веса — чувствительность глаза, лекция 2); C_b, C_r — отклонение синего и красного от яркости, 128 — «нет цвета».

исходное RGB



Y — яркость



C_b — «синева»



C_r — «краснота»



C_b, C_r в 2 раза меньше
(4:2:0), PSNR 40.6 дБ



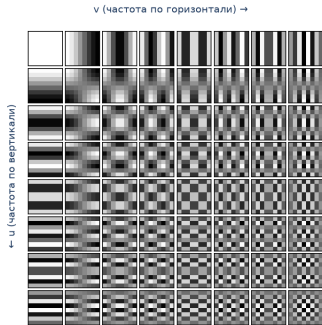
Глаз различает мелкие детали **яркости** гораздо лучше, чем мелкие детали **цвета**. Поэтому C_b, C_r хранят в 2 раза меньшего размера по каждой оси (**4:2:0**: в 4 раза меньше данных о цвете) — разницы почти не видно, а объём падает вдвое ещё до DCT. Это ФНЧ + прореживание, как в гауссовой пирамиде.

$$F[u, v] = \frac{1}{4} \alpha(u) \alpha(v) \sum_{m=0}^7 \sum_{n=0}^7 f[m, n] \cos \frac{(2m+1)u\pi}{16} \cos \frac{(2n+1)v\pi}{16}$$

- $f[m, n]$ — яркости блока 8×8 минус 128;
 $m, n = 0..7$ — строка и столбец;
- $u, v = 0..7$ — частоты; $F[u, v]$ — сколько в блоке узора (u, v) (справа);
- $\cos \frac{(2m+1)u\pi}{16}$ — значение узора в строке m : u полупериодов на 8 пикселей;
- $\alpha(0) = 1/\sqrt{2}$, $\alpha(k > 0) = 1$ — нормировка (обратимость без искажений).

Почему DCT, а не ДПФ

Коэффициенты **действительные**; блок неявно **отражается** (reflect), а не склеивается (wrap) — нет ложного шва.



$(0, 0)$ — ровный блок (средняя яркость);
вправо — вертикальные полосы всё
чаще, вниз — горизонтальные.

Шаг 3: квантование — здесь теряется информация

JPEG и вейвлеты

1) пиксели блока 8×8

59	62	62	57	53	49	23	106
57	60	58	51	48	27	29	132
56	52	46	40	28	17	56	145
48	43	37	18	13	17	45	143
37	34	19	16	19	22	33	119
30	36	37	38	40	42	47	49
53	56	59	62	63	64	66	67
74	76	78	78	78	79	80	82

2) DCT(блок — 128)

-590	-60	96	-80	56	-44	24	-10
-17	-5	50	-50	41	-31	20	-8
84	39	-62	24	-12	5	1	-1
-49	12	-35	23	-10	-3	3	-5
6	-4	18	-2	4	-10	11	-6
3	0	-1	3	-9	8	1	1
-2	3	-9	7	-9	10	-6	1
4	-7	9	-1	7	-1	-1	-3

3) таблица Q (качество 50)

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

4) round(DCT/Q): 19 не нулей из 64

-37	-5	10	-5	2	-1	0	0
-1	0	4	-3	2	-1	0	0
6	3	-4	1	0	0	0	0
-4	1	-2	1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

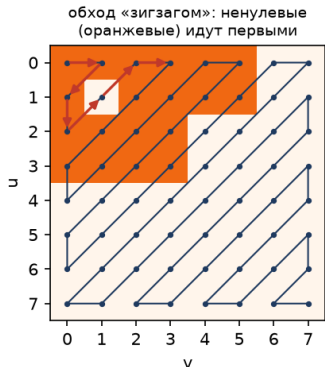
5) восстановленный блок

57	72	53	58	73	26	18	102
59	74	54	47	53	19	38	135
55	68	48	30	27	11	56	159
43	51	34	17	13	8	56	140
32	33	25	18	17	19	47	90
36	32	33	36	36	40	49	55
55	51	59	62	58	65	70	57
73	70	81	82	71	82	91	74

$$\hat{F}[u, v] = \text{round}\left(\frac{F[u, v]}{Q[u, v]}\right), \quad F'[u, v] = \hat{F}[u, v] \cdot Q[u, v]$$

- F — коэффициенты DCT (2), Q — **таблица шагов** квантования (3): маленькие шаги для низких частот, большие — для высоких;
- $\hat{F}$ — целые числа, которые пишутся в файл (4): из 64 осталось 19 ненулевых; F' — что получит декодер; ошибка $F - F'$ — до $Q/2$ на коэффициент (как $\Delta/2$ в лекции 1).

«Качество» JPEG масштабирует всю таблицу Q : качество 50 — стандартная таблица, 90 — шаги в 5 раз меньше, 10 — в 5 раз больше. Восстановленный блок (5) близок к исходному (1), но не равен ему.



Зигзаг. Коэффициенты читают по диагоналям от низких частот к высоким. Ненулевые оказываются **в начале**, а в конце — длинный хвост нулей:

-37 -5 -1 6 0 10 -5 4 3 -4 0 1 -4 ... 1 0 -1, а потом **37 нулей подряд** (блок со слайда про квантование).

RLE (кодирование длин серий): вместо 37 нулей пишут «дальше одни нули» — один символ EOB (end of block).

Хаффман: частые пары «(число нулей перед, величина)» получают короткие коды, редкие — длинные. Этот шаг **без потерь**: декодер восстановит $\hat{F}$ точно.

Коэффициент $F[0, 0]$ (средняя яркость блока) кодируют отдельно — как разность с соседним блоком: соседние блоки обычно похожи по яркости.

оригинал, 256 КБ (8 бит/пикс.)



Q=90: 58 КБ, 40.3 дБ



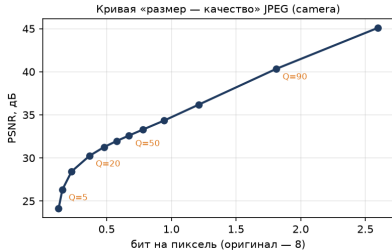
Q=50: 22 КБ, 32.6 дБ



Q=20: 12 КБ, 30.2 дБ



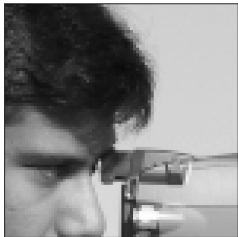
Q=5: 5 КБ, 26.3 дБ



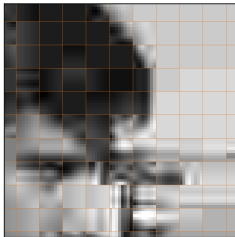
Сверху — фрагмент при разном качестве: подпись — размер файла и PSNR. **Слева** — кривая «размер-качество» (rate-distortion): бит на пиксель против PSNR.

До качества ≈ 50 (0.7 бит/пикс., сжатие в 11 раз, 33 дБ) потери почти не видны; дальше **каждый бит стоит всё дороже**: чтобы получить ещё +7.7 дБ (до Q=90), размер нужно увеличить в 2.6 раза. Ниже Q=20 качество падает обвально.

оригинал (80×80, увеличено)



JPEG Q=8: блоки 8×8 и «звон» у краёв



Оранжевая сетка — границы блоков 8×8 .

Что видно и откуда

- **Блочность** — каждый блок квантуется отдельно, на стыках скачки яркости.
- «Звон» у резких краёв — высокие частоты отрезаны: **эффект Гиббса** внутри блока.
- **Ровные плашки** — в блоке остался один $F[0, 0]$.
- **Расплывание цвета** — от 4:2:0.

Практические следствия

Маски, разметку и медицинские снимки — не в JPEG (PNG, TIFF). Не пересохранять: потери копятся.

```
import numpy as np
from scipy.fft import dctn, idctn

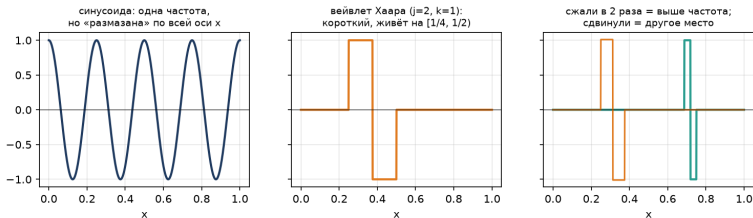
blk = I[200:208, 232:240] - 128.0
C = dctn(blk, norm="ortho") # шаг 2
Cq = np.round(C / Q50)      # шаг 3
print((Cq != 0).sum(), "из 64 не нули")

# декодер: обратно
rec = idctn(Cq * Q50, norm="ortho") + 128
print(np.abs(rec - 128 - blk).max())
```

Q50 — стандартная таблица 8×8 (слайд с квантованием); `norm="ortho"` — та самая нормировка $\frac{1}{4}\alpha(u)\alpha(v)$.

```
import cv2
for q in [90, 50, 20, 5]:
    p = [cv2.IMWRITE_JPEG_QUALITY, q]
    ok, buf = cv2.imencode(".jpg", I8, p)
    dec = cv2.imdecode(buf, 0) # серое
    bpp = 8 * len(buf) / I8.size
    err = dec.astype(float) - I8
    mse = np.mean(err ** 2)
    psnr = 10 * np.log10(255**2 / mse)
    print(q, round(bpp, 2), round(psnr, 1))
```

`imencode` сжимает **в память**, без файла — удобно мерить размер. `I8` — изображение `uint8`; перед разностью — в `float`, иначе `uint8` «переполнится».



Синусоида тянется через **всю** картинку: Фурье говорит, **какая** частота есть, но не **где**. DCT обходит это блоками — отсюда блочность. **Вейвлет** — короткая волна: есть и частота (ширина), и место (сдвиг).

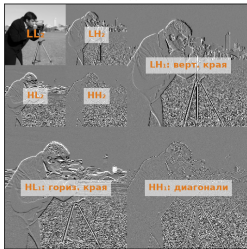
$$a_i = \frac{x_{2i} + x_{2i+1}}{\sqrt{2}}, \quad d_i = \frac{x_{2i} - x_{2i+1}}{\sqrt{2}}$$

- **Хаар**, простейший вейвлет: соседние пары отсчётов;
- a_i — **среднее** (грубая копия, вдвое короче), d_i — **разность** (детали); $\sqrt{2}$ — чтобы энергия сохранялась;
- повторяем для a — следующий уровень.

исходное 512×512



Хаар, 2 уровня



Хаар применяем к строкам, затем к столбцам: 4 части по $\frac{1}{4}$ размера. **LL** — уменьшенная копия; **LH, HL, HH** — детали по направлениям. LL раскладываем дальше. Почти все детали ≈ 0 (серые) — их дешево хранить. Это **пирамида без лишних пикселей**: всего ровно MN чисел.

оригинал (фрагмент)



DCT 8×8, 3% коэфф.: 28.9 дБ
блоки



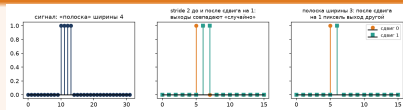
вейвлет db4, 3% коэфф.: 29.8 дБ
без блоков, мягче



Оставим **3%** самых больших коэффициентов. Блочный DCT — видны блоки; вейвлет (db4, 4 уровня) — блоков нет, ошибка «размазана» и выше PSNR. На этом построен **JPEG 2000** (кино, медицина DICOM, спутниковые снимки) — но обычный JPEG проще и всё ещё повсюду.

Где эта идея живёт в глубоком обучении

- **Stride и пулинг в CNN** — прореживание **без сглаживания**. Сеть меняет ответ при сдвиге на 1 пиксель (справа). Лекарство — blur перед stride (Zhang, 2019).
- **FPN, U-Net** — обучаемые пирамиды признаков разных масштабов.
- **Свёртка через FFT** — быстрые свёртки с большими ядрами; FNO учит прямо **маску в частотах**, FNet заменяет attention на FFT.
- У обученных CNN фильтры первого слоя похожи на **волны Габора** — локальные «синусоиды», как вейвлеты.



Подробно — в курсе «Глубокое обучение в задачах компьютерного зрения». Здесь только связь, без деталей.

1. Изображение — **сумма волн** (решёток); у волны есть частота, направление, амплитуда и фаза. Низкие частоты — форма и освещение, высокие — края, текстура, шум.
2. ДПФ находит амплитуду и фазу каждой волны; спектр смотрят как $\log(1 + |F|)$ со сдвигом в центр. ДПФ считает картинку периодической — отсюда «крест»; лекарство — окно. Фаза кодирует **где** стоят объекты; FFT — за $O(N \log N)$.
3. **Теорема о свёртке**: свёртка = умножение спектров. Вох звенит, потому что его спектр (sinc) уходит в минус; спектр Гаусса — Гаусс.
4. ФНЧ сглаживает, ФВЧ оставляет края, режекторный убирает периодическую помеху; маски должны быть **гладкими**.
5. Гауссова пирамида — всё более грубые копии; лапласова — полосы частот без потерь; блендинг — смешивание по уровням.
6. JPEG: YCbCr + 4:2:0, DCT блоков 8×8 , квантование (главная потеря), зигзаг + энтропийный код. Вейвлеты — локальные волны без блоков (JPEG 2000).

Основная

Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., гл. 3.4 (Fourier transforms), 3.5 (Pyramids and wavelets) — <https://szeliski.org/Book/>.

Gonzalez, Woods, *Digital Image Processing*, 4th ed., гл. 4 (Filtering in the frequency domain), гл. 7 (Wavelets), гл. 8 (Image compression).

Первоисточники и дополнительно

Burt, Adelson, *A multiresolution spline with application to image mosaics*, ACM TOG, 1983 — блендинг пирамидами.

Oppenheim, Lim, *The importance of phase in signals*, Proc. IEEE, 1981 — опыт с фазой.

Wallace, *The JPEG still picture compression standard*, IEEE Trans. Consumer Electronics, 1992.

Zhang, *Making convolutional networks shift-invariant again*, ICML 2019 — сглаживание перед stride в CNN.

Изображения яблока и апельсина — из OpenCV `samples/data` (лицензия Apache 2.0); остальные иллюстрации построены по `skimage.data` скриптом `scripts/make_figs_L04.py`.

1. Что такое амплитуда, частота и фаза синусоиды? Какая самая высокая частота возможна на пиксельной сетке и почему?
2. Запишите 1D-ДПФ. Что такое $X[0]$? Почему спектр действительного сигнала симметричен?
3. Как выглядит в спектре решётка (u, v) ? Как изменится спектр, если повернуть изображение на 30° ?
4. Откуда в спектре фотографии яркий «крест» по осям и как от него избавиться?
5. Опишите опыт с обменом фаз. Что он показывает?
6. Сформулируйте теорему о свёртке. Почему box-фильтр «звенит», а Гаусс — нет? Ответьте через спектры.
7. Сравните идеальный, Баттерворта и гауссов ФНЧ. Как получить ФВЧ и полосовой фильтр?
8. Почему FFT быстрее ДПФ «в лоб»? Когда свёртку выгоднее считать через FFT, а когда напрямую?
9. Определите гауссову и лапласову пирамиды. Почему лапласова восстанавливает изображение точно?
10. Перечислите шаги JPEG. На каких шагах теряется информация? Откуда блочность и звон? Чем вейвлеты лучше блочного DCT?